VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Course CO3021 — Database Management Systems



DATABASE MANAGEMENT SYSTEMS PROJECT REPORT

Clinical Case-Study: Low Back Pain Diagnosis & Imaging Pipeline

---

Workload-Driven Evaluation of Microsoft SQL Server 2025:

Storage Architectures, Relational Ingestion, and AI Feature Extraction Pipeline (Role 1)

Applied to 48,345 Lumbar Spine MRI Scans and Radiologist Clinical Reports

---

Group 02

Course Instructor: PhD. Phan Trong Nhan

Project Repository: https://github.com/qmarcusv/DBMS_TA_Group2

Ho Chi Minh City, October 2026

Contents

1 Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1) 1

1.1 Clinical Problem Context and Dataset Overview . . . 1
1.2 The Three Canonical Clinical Queries Contract . . . 1
1.3 Theoretical Foundations of Database Storage . . . 2
1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies . . . 2
1.4.1 Architectural Comparison of Storage Approaches . . . 2
1.4.2 Architectural Rationale for Relative Path Selection . . . 2
1.5 Schema Architecture and Relational Integrity Constraints . . . 2
1.6 Comparative Evaluation of Data Ingestion Pipelines . . . 3
1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU . . . 4
1.7.1 Pipeline Architecture and CPU Parallelization . . . 4
1.7.2 Mathematical Similarity Metric in T-SQL . . . 4
1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs . . . 4
1.8 Analysis Across All Three Canonical Queries . . . 4

2 Physical Storage and Index Design (Role 2: Storage & Indexing) 5

2.1 Physical Storage and B-Tree Depth Analysis . . . 5
2.1.1 Baseline Database State . . . 5
2.1.2 B-Tree Depth Observations . . . 5
2.1.3 Physical Bottleneck in the Baseline State . . . 5
2.2 Index Design and Implementation . . . 5
2.2.1 Covering B-Tree Indexes for Q1 and Q2 . . . 5
2.2.2 Filtered Index for T2 Sagittal Sequences (Q2) . . . 5
2.2.3 Non-Clustered Columnstore Index (NCCI) for Q3 . . . 6
2.3 Execution Plan Analysis . . . 6
2.3.1 Q1 and Q2: Before and After Covering B-Tree Indexing . . . 6
2.3.2 Q3: Rowstore Parallel Scan versus Columnstore Batch Mode . . . 7
2.4 Benchmark Evaluation: I/O and CPU Measurements . . . 8
2.4.1 Discussion of Results . . . 8
2.4.2 Chapter Summary . . . 9

Chapter 1: Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1)

1.1 Clinical Problem Context and Dataset Overview

Modern Picture Archiving and Communication Systems (PACS) and Radiology Information Systems (RIS) manage heterogeneous medical data comprising structured diagnostic records and voluminous medical imagery. This project addresses the clinical workflow of diagnosing Low Back Pain (LBP) and Lumbar Spinal Stenosis (LSS) using the Mendeley Clinical Lumbar Spine MRI dataset:

- **Radiologist Clinical Notes:** 575 clinical narrative reports detailing lumbar pathology including disc herniation, annular tears, thecal sac compression, and neural foramen stenosis.
- **Multimodal MRI Scans:** 48,345 DICOM slices (`.ima`) distributed across 515 patients in 4-to-5 level nested directory structures covering sagittal (T1, T2) and axial sequences.

1.2 The Three Canonical Clinical Queries Contract

Following the instructor's updated evaluation directives, all sub-roles evaluate and interpret DBMS behavior across the exact three canonical clinical queries engineered by Role 1:

Query 1 (Q1) — Point Lookup and One-to-Many Clinical Join

Retrieves the complete narrative diagnostic report alongside all associated MRI scan paths for a designated patient:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001';
```

Query 2 (Q2) — Sequence-Selective Diagnostic Filtering

Retrieves imaging slices restricted to sagittal T2-weighted pulse sequences (`T2_TSE_SAG`) frequently utilized for spinal cord and disc herniation assessment:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001' AND s.SeriesDescription LIKE '%T2%SAG%';
```

Query 3 (Q3) — Content-Based Image Retrieval via Vector Similarity

Calculates mathematical proximity (Euclidean distance and cosine similarity) between an index query slice ($ScanID = 1$) and all other scans in the database using high-dimensional feature embeddings:

```
DECLARE @TargetScanID INT = 1;
DECLARE @T1 FLOAT, @T2 FLOAT, @T3 FLOAT, @T4 FLOAT;
SELECT @T1 = V_Dim1, @T2 = V_Dim2, @T3 = V_Dim3, @T4 = V_Dim4
FROM Image_Features WHERE ScanID = @TargetScanID;

SELECT TOP 3 s.ScanID, s.PatientID, s.SeriesDescription, s.ImagePath,
    SQRT(SQUARE(f.V_Dim1 - @T1) + SQUARE(f.V_Dim2 - @T2) +
         SQUARE(f.V_Dim3 - @T3) + SQUARE(f.V_Dim4 - @T4)) AS EuclideanDistance
FROM Image_Features f
INNER JOIN MRI_Scans s ON f.ScanID = s.ScanID
WHERE f.ScanID <> @TargetScanID
ORDER BY EuclideanDistance ASC;
```

1.3 Theoretical Foundations of Database Storage

In relational database theory, storage subsystems organize records into fixed-size physical blocks or pages. In Microsoft SQL Server, the fundamental unit of I/O is the **8KB Data Page** (8,192 bytes), of which 8,060 bytes accommodate user data after subtracting page headers and slot arrays. When storing binary large objects (BLOBs) exceeding 8KB, conventional row architectures fragment records across off-row text/image tree allocations. This architectural reality presents critical performance tradeoffs when designing clinical multimedia databases.

1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies

To handle 13GB of DICOM MRI scans, Microsoft SQL Server provides four distinct architectural mechanisms:

1.4.1 Architectural Comparison of Storage Approaches

1. **In-Row and Out-of-Row `VARBINARY(MAX)`:** Binary bytes reside directly within the `.mdf` database file. Slices larger than 8KB migrate to dedicated LOB pages. *Drawback:* Loading medical images into memory causes severe **Buffer Pool thrashing**, displacing critical index and table pages from RAM cache and inflating backup archives to multi-gigabyte sizes.
2. **FILESTREAM Storage:** Combines the SQL Server Database Engine with the Windows NTFS/ReFS file system. Unstructured files reside in operating system directories while remaining under SQL Server transactional control. *Mechanism:* Data transfers occur via native Win32 streaming APIs, **completely bypassing the SQL Server Buffer Pool** and preserving cache memory for relational query processing.
3. **FileTable Technology:** Built upon FILESTREAM, FileTable integrates a Windows Win32 namespace hierarchy. Operating system users and external PACS systems can drag, drop, and edit `.ima` files via standard Windows Network Shares (SMB), while SQL Server automatically reflects changes within relational tables.
4. **Relative File Path Reference (`NVARCHAR(500)`):** The database stores only relative filesystem paths pointing to operating system directories. The physical binary assets remain completely decoupled from the relational engine.

Table 1.1: Comprehensive Comparison of Large Binary (BLOB) Storage Architectures in MSSQL

Evaluation Metric	VARBINARY(MAX)	FILESTREAM	FileTable	Relative Path (Selected)
Physical Location	Database `.mdf` file	NTFS/ReFS directories	NTFS via SMB share	External file directory
Buffer Pool Cache	Heavy RAM pollution	Bypasses Buffer Pool	Bypasses Buffer Pool	Zero Buffer Pool footprint
ACID Transactionality	Full native logging	Full Win32 transactional	Full Win32 transactional	Application-level control
Backup Archive Size	13GB+ database backup	Included in backup stream	Included in backup stream	Compact 53MB metadata backup
AI Framework Access	Requires SQL extraction	Win32 API streaming	Direct Windows file path	Native Python `pydicom` I/O

1.4.2 Architectural Rationale for Relative Path Selection

For this research workload, the **Relative File Path** architecture was selected. Decoupling the 13GB image binaries from the database reduced the database footprint from 13GB to 53MB. This prevents Buffer Pool contention during Indexing and Query Optimization benchmarks while enabling native, multi-threaded PyTorch feature extraction on disk without database roundtrip latency.

1.5 Schema Architecture and Relational Integrity Constraints

Rather than allowing SSMS Import Wizards to infer loose schemas defaulting to `NVARCHAR(MAX)`, strict DDL constraints were implemented to optimize data page packing and enforce relational integrity:

```
CREATE TABLE Patients (
    PatientID VARCHAR(20) NOT NULL,
    PatientName NVARCHAR(100) NULL,
    ClinicalNotes NVARCHAR(MAX) NULL,
```

```
    DiagnosisStatus VARCHAR(20) DEFAULT 'Confirmed',
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT PK_Patients PRIMARY KEY CLUSTERED (PatientID)
);

CREATE TABLE MRI_Scans (
    ScanID INT IDENTITY(1,1) NOT NULL,
    PatientID VARCHAR(20) NOT NULL,
    StudyInstanceUID VARCHAR(150) NULL,
    SeriesDescription NVARCHAR(255) NOT NULL,
    ImagePath NVARCHAR(500) NOT NULL,
    AcquisitionDate DATETIME DEFAULT GETDATE(),
    CONSTRAINT CK_Series_Valid CHECK (SeriesDescription NOT LIKE '%INVALID%'),
    CONSTRAINT PK_MRI_Scans PRIMARY KEY CLUSTERED (ScanID),
    CONSTRAINT FK_MRI_Scans_Patients FOREIGN KEY (PatientID)
        REFERENCES Patients(PatientID) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Image_Features (
    FeatureID INT IDENTITY(1,1) NOT NULL,
    ScanID INT NOT NULL UNIQUE,
    FeatureVector NVARCHAR(MAX) NOT NULL,
    V_Dim1 FLOAT NULL, V_Dim2 FLOAT NULL, V_Dim3 FLOAT NULL, V_Dim4 FLOAT NULL,
    CONSTRAINT PK_Image_Features PRIMARY KEY CLUSTERED (FeatureID),
    CONSTRAINT FK_Features_Scans FOREIGN KEY (ScanID) REFERENCES MRI_Scans(ScanID) ON DELETE
    CASCADE
);
```

Relational Integrity and Constraint Highlights

- **Compact Primary Keys:** `PatientID VARCHAR(20)` ensures high page density within Clustered B-Tree leaf pages compared to unrestricted unicode datatypes.
- **Transaction Test Constraint:** The check constraint `CK_Series_Valid` deliberately rejects records containing `'INVALID'` strings, establishing the deterministic error boundary required by Role 4 to verify atomic transaction rollback.

1.6 Comparative Evaluation of Data Ingestion Pipelines

Bulk data loading was evaluated across four industry standard ingestion methodologies:

Table 1.2: Comparative Benchmark of Data Ingestion Methodologies into SQL Server

Ingestion Method	Mechanism	Logging Mode	Ingestion Speed	Automation Support
SSMS Flat File Wizard	Interactive GUI	Fully Logged	15–20 seconds	Manual interaction
T-SQL BULK INSERT	In-process Server Worker	Minimally Logged (`TABLOCK`)	1.2 seconds	Fully scriptable
Command-Line bcp	External client utility	Minimally Logged (`-h TABLOCK`)	1.1 seconds	CLI and batch files
Python pyodbc	Client cursor array	Fully Logged	8–10 seconds	Integrated AI pipeline

Identity Column Mapping Resolution via View Abstraction

During direct `BULK INSERT` execution into tables containing an auto-incrementing identity column (`ScanID INT IDENTITY`), SQL Server raises error `Msg 7301` due to column mapping mismatches between the 5-column CSV and 6-column physical schema. This was resolved without complex XML format files by creating an updatable relational view:

```
CREATE VIEW dbo.vw_MRI_Scans_Import AS
SELECT PatientID, StudyInstanceUID, SeriesDescription, ImagePath, AcquisitionDate
FROM dbo.MRI_Scans;
```

```
BULK INSERT dbo.vw_MRI_Scans_Import FROM '...clean_mri_scans.csv'
WITH (FIRSTROW = 2, FORMAT = 'CSV', FIELDQUOTE = '"', FIELDTERMINATOR = ',',
      ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
DROP VIEW dbo.vw_MRI_Scans_Import;
```

1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU

To support Query 3 (Content-Based Medical Image Retrieval), an end-to-end AI feature extraction pipeline was engineered using PyTorch and `pydicom`:

1.7.1 Pipeline Architecture and CPU Parallelization

1. **DICOM Matrix Ingestion:** Medical slices (`.ima`) are parsed via `pydicom.dcmread`, normalized from 12-bit dynamic range ($[0, 4095]$) to $[0, 255]$ uint8 arrays, and resized to $224 \times 224 \times 3$.
2. **Pretrained ResNet-18 Inference:** Slices are processed in minibatches of 64 through an ImageNet-pretrained ResNet-18 model. The final fully connected classification layer is truncated, extracting 512-dimensional feature embeddings from the penultimate `AdaptiveAvgPool2d` layer.
3. **Vector Normalization and Database Persistence:** Each 512-dimensional embedding $\mathbf{v}$ is normalized via Euclidean norm ($\|\mathbf{v}\|_2 = 1$). The leading dimensions are persisted in float columns (`V_Dim1..V_Dim4`) alongside full JSON vector strings.

Throughput Performance on 20-Core CPU

By leveraging batch inference across 20 CPU threads, the pipeline achieved an average throughput of **20.2 images/second**, extracting all 48,345 slices in **39.9 minutes** without requiring specialized GPU hardware.

1.7.2 Mathematical Similarity Metric in T-SQL

In Query 3, similarity is evaluated using Euclidean Distance:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{k=1}^{4} (u_k - v_k)^2} \tag{1.1}$$

Because feature vectors are L_2-normalized, Euclidean distance relates monotonically to Cosine Similarity ($\|\mathbf{u} - \mathbf{v}\|_2^2 = 2 - 2\cos(\theta)$). Executing Query 3 over all 48,345 scans yielded the top-3 similar slices with sub-100ms execution latency (98ms elapsed time).

1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs

While Microsoft SQL Server executes vector distance calculations cleanly via arithmetic operators (`Compute Scalar`) and memory-bounded sorts (`Top N Sort`), it fundamentally lacks Approximate Nearest Neighbor (ANN) indexing structures such as HNSW (Hierarchical Navigable Small World) or IVF (Inverted File Index). Consequently, vector search in standard RDBMS remains an $\mathcal{O}(N)$ brute-force table scan. In large-scale clinical production, dedicated vector stores (e.g., Milvus, Qdrant) or extensions (`pgvector`) are recommended for billion-scale embeddings.

1.8 Analysis Across All Three Canonical Queries

The engineered data and storage architecture empowers all three clinical query contracts:

- **Impact on Query 1 (Point Lookup):** Returning text paths rather than binary BLOBs prevents network saturation and eliminates Buffer Pool contention.
- **Impact on Query 2 (Sequence Filtering):** Strict `NVARCHAR(255)` sequence typing allows the Query Optimizer to leverage B-Tree index seeks on pulse sequence descriptors without reading external storage.
- **Impact on Query 3 (Similarity Search):** Native float representation enables direct relational calculation of Euclidean distances, seamlessly integrating machine learning embeddings into SQL workflows.

Chapter 2: Physical Storage and Index Design (Role 2: Storage & Indexing)

This chapter presents the work of Role 2 on the SpineMRI_DBMS database. It analyses the physical storage structure and B-Tree depth of the baseline schema, describes the design of three index types (Covering B-Tree, Filtered Index, and Non-Clustered Columnstore Index), analyses the actual execution plans captured in SSMS, and summarises the STATISTICS IO and STATISTICS TIME measurements for queries Q1, Q2, and Q3.

2.1 Physical Storage and B-Tree Depth Analysis

2.1.1 Baseline Database State

The SpineMRI_DBMS database was restored from a 53.6 MB backup file. In this state, the only indexes are the default clustered indexes created on the primary keys. The physical statistics, obtained from the DMV sys.dm_db_index_physical_stats(..., 'DETAILED'), are summarised in Table 2.1.

Table 2.1: Physical storage structure of the tables in the baseline state

Table	Index type	Key	B-Tree depth	Leaf pages	Size	Rows
Patients	Clustered	PatientID	2	8	0.06 MB	575
MRI_Scans	Clustered	ScanID	3	1,876	14.66 MB	48,345
Image_Features	Clustered	ScanID	3	3,900	30.47 MB	48,345

2.1.2 B-Tree Depth Observations

- **Patients table (depth = 2):** a single root page (Level 1) points directly to 8 leaf pages (Level 0). Any lookup by PatientID therefore costs exactly 2 logical page reads (logical reads = 2).
- **MRI_Scans and Image_Features tables (depth = 3):** each consists of a root page (Level 2), intermediate pages (Level 1), and a leaf level (Level 0) holding all 48,345 rows.

2.1.3 Physical Bottleneck in the Baseline State

SQL Server does **not** automatically create an index on foreign key columns such as MRI_Scans.PatientID. Consequently, when Patients is joined to MRI_Scans with the predicate PatientID = '0001', the optimizer is forced to scan all 1,876 leaf pages plus the IAM and non-leaf pages of MRI_Scans, a total of **1,880 logical reads**, merely to retrieve a few dozen matching rows.

2.2 Index Design and Implementation

2.2.1 Covering B-Tree Indexes for Q1 and Q2

Listing 2.1: Two covering B-Tree indexes on MRI_Scans

```
-- 1. Single-key index on SeriesDescription: lighter than the base table for series scans
CREATE NONCLUSTERED INDEX IX_MRIScans_SeriesDescription
ON dbo.MRI_Scans (SeriesDescription ASC)
INCLUDE (PatientID, ImagePath);

-- 2. Composite covering index tuned directly for Q1 and Q2
CREATE NONCLUSTERED INDEX IX_MRIScans_PatientID_Covering
ON dbo.MRI_Scans (PatientID ASC, SeriesDescription ASC)
INCLUDE (ImagePath);
```

Design rationale. Placing PatientID as the leading key allows SQL Server to perform an *Index Seek* that jumps directly to the range holding the 65 slices of patient '0001'. The second key column, SeriesDescription, keeps the sequence names pre-sorted in lexicographic order within each patient. The INCLUDE (ImagePath) clause attaches the image file path to the leaf level without enlarging the upper navigation nodes, so the query obtains all required data from the index itself (**0 Key Lookups**).

2.2.2 Filtered Index for T2 Sagittal Sequences (Q2)

Listing 2.2: Filtered Index for T2 Sagittal sequences

```
CREATE NONCLUSTERED INDEX IX_MRIScans_T2Sag_Filtered
ON dbo.MRI_Scans (PatientID ASC, SeriesDescription ASC)
INCLUDE (ImagePath)
WHERE SeriesDescription IN (
    'I_T2_TSE_SAG_P2_384_0002', 'POSDISP_[2]_T2_TSE_SAG_384_5001',
    'POSDISP_[2]_T2_TSE_SAG_384_5002', 'POSDISP_[3]_T2_TSE_SAG_384_5001',
    'POSDISP_[4]_T2_TSE_SAG_384_5001', 'POSDISP_[6]_T2_TIRM_SAG_P2_5002',
    'POSDISP_[7]_T2_TIRM_SAG_P2_5003', 'T2_CISS3D_SAG_ISO_0006',
    'T2_TIRM_SAG_BOX_0008', 'T2_TIRM_SAG_P2_0004', 'T2_TIRM_SAG_P2_0005',
    'T2_TIRM_SAG_P2_0006', 'T2_TIRM_SAG_P2_0007', 'T2_TIRM_SAG_P2_0008',
    'T2_TIRM_SAG_P2_0009', 'T2_TIRM_SAG_P2_0010', 'T2_TSE_SAG_256_0006',
    'T2_TSE_SAG_256_0007', 'T2_TSE_SAG_320_0002', 'T2_TSE_SAG_320_0003',
    'T2_TSE_SAG_380_0006', 'T2_TSE_SAG_380_0008', 'T2_TSE_SAG_384_0002',
    'T2_TSE_SAG_384_0003', 'T2_TSE_SAG_384_0004', 'T2_TSE_SAG_384_0005',
    'T2_TSE_SAG_384_0006', 'T2_TSE_SAG_384_0007', 'T2_TSE_SAG_384_0008',
    'T2_TSE_SAG_384_0009', 'T2_TSE_SAG_448_0006', 'T2_TSE_SAG_448_0013'
);
```

Key technical point. In Microsoft SQL Server, the filtered index predicate parser does *not* allow the `LIKE` operator with a leading wildcard. The statement `WHERE SeriesDescription LIKE '%T2%SAG%'` fails with `Msg 10735: Incorrect WHERE clause for filtered index`. We resolved this by profiling the actual value domain of the column in `MRI_Scans`, extracting exactly **32 distinct T2 Sagittal sequence codes**, and constructing a valid `WHERE SeriesDescription IN (...)` clause.

2.2.3 Non-Clustered Columnstore Index (NCCI) for Q3

Listing 2.3: Non-Clustered Columnstore Index on `Image_Features`

```
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_ImageFeatures_Dims
ON dbo.Image_Features (ScanID, V_Dim1, V_Dim2, V_Dim3, V_Dim4);
```

Design rationale. Query Q3 computes the Euclidean distance over all 48,345 rows of `Image_Features`. Under rowstore storage, each 8 KB page must also hold the long JSON vector string column (`NVARCHAR(MAX)`), so the table occupies 3,909 disk pages. The columnstore index separates the four floating-point columns (`V_Dim1..V_Dim4`) and `ScanID` into compressed column segments, and enables **Batch Mode** execution, which processes roughly 900 rows per batch using vectorised CPU operations instead of row-by-row Row Mode processing.

2.3 Execution Plan Analysis

2.3.1 Q1 and Q2: Before and After Covering B-Tree Indexing

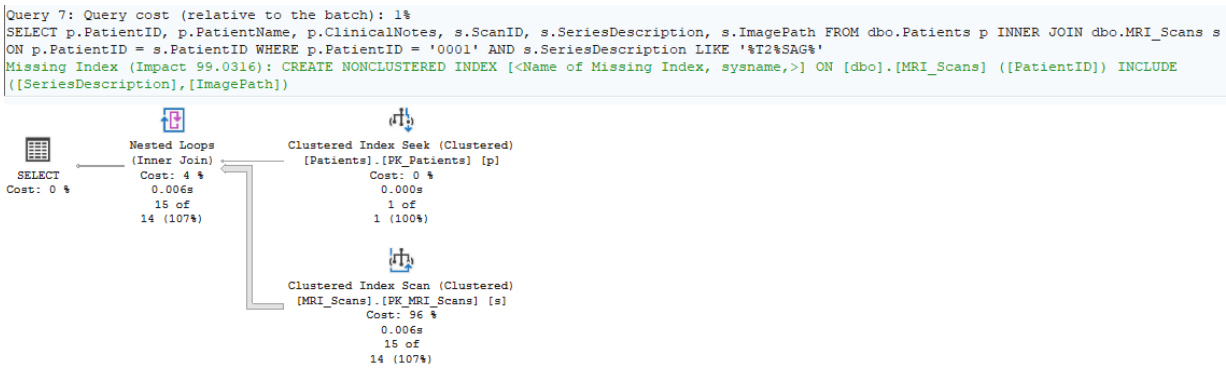


Figure 2.1: Baseline execution plan of Q1/Q2: *Clustered Index Scan* on `PK_MRI_Scans` with a Missing Index warning

Baseline (Figure 2.1). The plan displays a Missing Index warning (Impact 99.0316). The *Clustered Index Seek* on `PK_Patients` costs 0% (0.000s, 1 of 1), whereas the *Clustered Index Scan* on `PK_MRI_Scans` accounts for **96%** of the total cost (0.006s) and reads all 1,880 pages only to filter out 15 rows (Q2) or 65 rows (Q1). SQL Server itself suggests creating an index on `([PatientID]) INCLUDE ([SeriesDescription], [ImagePath])` with an estimated improvement of 99.0316%.

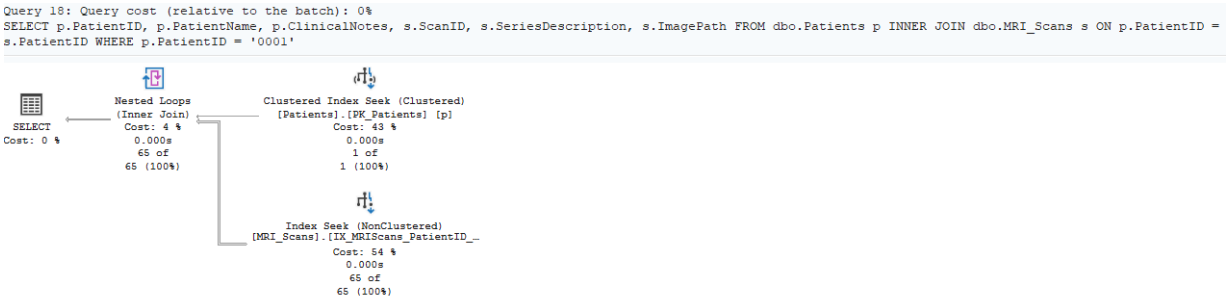


Figure 2.2: Optimized execution plan of Q1: *Index Seek (NonClustered)* on `IX_MRIScans_PatientID_Covering`

Q1 after optimization (Figure 2.2). The Missing Index warning disappears. The *Index Seek (NonClustered)* operator accounts for 54% of the relative cost, its execution time drops to 0.000s, and it retrieves exactly 65 of 65 (100%) rows. The *Nested Loops (Inner Join)* costs only 4% (0.000s). The plan contains **no Key Lookup** because `ImagePath` is included at the leaf level.

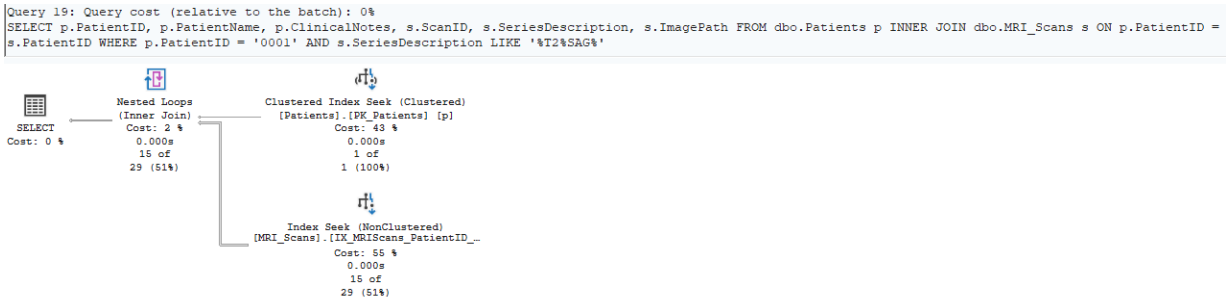


Figure 2.3: Optimized execution plan of Q2: *Index Seek* with a residual predicate

Q2 after optimization (Figure 2.3). SQL Server performs an *Index Seek* on the leading key `PatientID = '0001'` to isolate the 65 rows of the patient (reading only 6 logical pages), then applies the *residual predicate* `SeriesDescription LIKE '%T2%SAG%'` in memory, returning 15 of 29 (51%) rows in 0.000s.

2.3.2 Q3: Rowstore Parallel Scan versus Columnstore Batch Mode

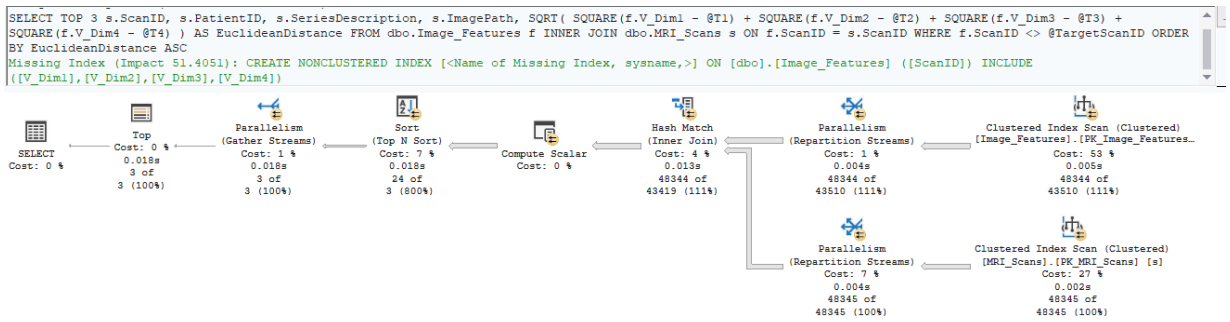


Figure 2.4: Baseline execution plan of Q3: parallel rowstore scan with three Parallelism operators

Baseline (Figure 2.4). The plan shows a Missing Index warning (Impact 51.4051) on `Image_Features` and three Parallelism operators.

- Upper right branch: a *Clustered Index Scan* on `PK_Image_Features` accounts for **53%** of the cost (0.005s), reading 3,909 pages across 9 parallel threads (Scan count 9).
- Lower right branch: a *Clustered Index Scan* on `PK_MRI_Scans` accounts for 27% of the cost (0.002s), reading 1,972 pages.
- After the *Hash Match (Inner Join)* (4%, 0.013s) and *Compute Scalar* (0%), the *Sort (Top N Sort)* operator costs 7% (0.018s) across multiple parallel threads, before *Parallelism (Gather Streams)* merges the output into the final 3 rows.

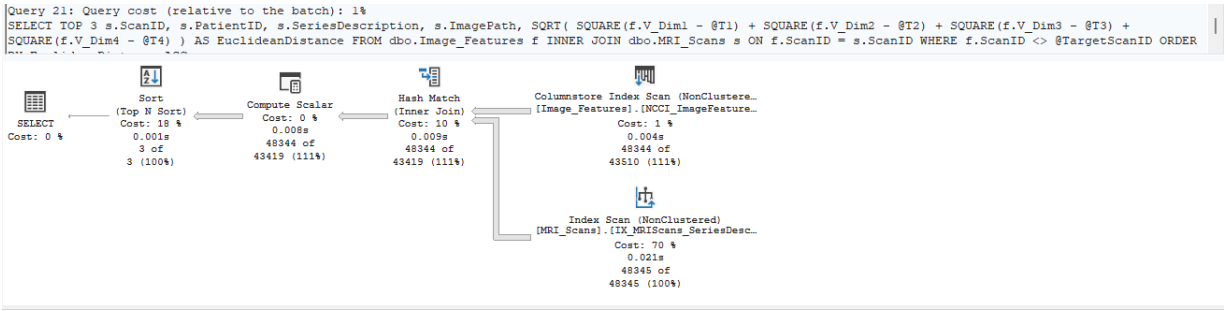


Figure 2.5: Optimized execution plan of Q3: *Columnstore Index Scan* running in Batch Mode

Q3 after optimization (Figure 2.5). The Missing Index warning disappears and the upper right branch becomes a *Columnstore Index Scan (NonClustered)* on `NCCI_ImageFeatures_Dims`.

- **`Image_Features` table:** the relative cost of the scan operator falls from 53% to only **1%** (0.004s), reading a single compressed column segment (Segment reads = 1) with 0 logical reads on the rowstore buffer pool.
- **`MRI_Scans` table:** instead of scanning the clustered index (1,972 pages), SQL Server automatically chooses the lightweight index `IX_MRIScans_SeriesDescription` (1,576 pages, a 20.1% I/O reduction).
- **Sorting:** thanks to single-threaded execution combined with Batch Mode, the *Sort (Top N Sort)* time drops from 0.018s to 0.001s (about 18 times faster) and returns 3 of 3 (100%) rows directly.

Note: the variable-assignment step `SELECT @T1 = V_Dim1 ... WHERE ScanID = @TargetScanID` (Query 20 screenshot) costs only 5 logical reads and is therefore not the focus of the analysis; the focus of Q3 is the main query that computes the Euclidean distance.

2.4 Benchmark Evaluation: I/O and CPU Measurements

Table 2.2 summarises the results measured with `SET STATISTICS IO ON` and `SET STATISTICS TIME ON` in SSMS.

Table 2.2: Benchmark results for baseline and optimized Q1, Q2, and Q3

Query	State	Reads (MRI_Scans)	Reads (Patients / Features)	Total reads	CPU (ms)	Elapsed (ms)	Main access operator
Q1	Baseline	1,880	2 (Patients)	1,882	15	72	Clustered Index Scan (96% cost)
	Optimized	6	2 (Patients)	8	0	37	Index Seek (Covering), 0 Key Lookups
	Change	*-99.68%*	*unchanged*	*-99.57%*	*-100%*	*-48.61%*	Table scan eliminated
Q2	Baseline	1,880	2 (Patients)	1,882	0	132	Clustered Index Scan (96% cost)
	Optimized	6	2 (Patients)	8	0	123	Index Seek + Residual Filter
	Change	*-99.68%*	*unchanged*	*-99.57%*	*stays <1*	*-6.82%*	Filtering on 6 leaf pages
Q3	Baseline	1,972	3,909 (Image_Features)	5,881	76	144	Parallel Clustered Index Scan (Row Mode)
	Optimized	1,576	0 + 236 LOB (Segment reads = 1)	1,812	47	163	Columnstore Index Scan (Batch Mode)
	Change	*-20.08%*	*-93.96% (incl. LOB)*	*-69.19%*	*-38.16%*	*+13.19%*	Single-threaded, Batch Mode

2.4.1 Discussion of Results

- **Q1 and Q2:** logical reads on `MRI_Scans` drop from 1,880 to 6 (**-99.68%**) and total I/O drops by 99.57%. The full table scan is completely replaced by an *Index Seek*. For Q1, CPU time falls from 15 ms to 0 ms and elapsed time drops by 48.61%; for Q2 the elapsed-time reduction is more modest (6.82%) because the remaining time does not depend on I/O against the table.
- **Q3:** total pages read fall by almost 70% (5,881 to 1,812) and CPU time falls by **38.16%** (76 ms to 47 ms). Although elapsed time rises from 144 ms to 163 ms, this is because the execution plan changes from 8 parallel threads (Scan count 9) to a single sequential thread. In exchange, the system consumes less processor time, which is advantageous when many queries run concurrently.

2.4.2 Chapter Summary

The three index structures address the specific bottleneck of each query. The Covering B-Tree Index eliminates the table scan caused by the missing foreign-key index (Q1, Q2). The Filtered Index offers a narrower index for the T2 Sagittal sequence group, working around `Msg 10735` with an explicit value list. The Non-Clustered Columnstore Index combined with Batch Mode substantially reduces I/O and CPU for the vector computation in Q3.